Description Submission view

**Available from**: Cuma, 27 Mart 2026, 9:00 AM
**Due date**: Cuma, 27 Mart 2026, 10:30 AM
**Requested files**: AirportManager.java (Download)
**Type of work**: Individual work

AIRPORT GATE MANAGER SYSTEM
Please read this manual carefully before beginning your implementation in the Moodle VPL environment. You are expected to build several classes that interact with each other to manage flights at an airport terminal.

System Architecture and Class Overview
Your system will model different types of flights and an airport to manage them. You will use an inheritance hierarchy where a general flight class shares its properties with more specific flight types.
Deliverables
Submit a single Java file named AirportManager.java via LMS System (Moodle).
1. Class: Flight (Base Class) - abstract
This is the parent class representing a generic flight.
● Attributes:
○ An integer representing the flight code.
○ An integer representing the minutes remaining until departure.
● Constructor:
○ Takes two parameters: int flightCode and int minutesToDeparture to initialize the attributes. (Note: These values can initially be 0, or even logically negative if a flight is already considered departed).
● Methods:
○ Implement standard getter and setter methods for both attributes.
■ getFlightCode() returning an int.
■ setFlightCode(int flightCode) returning void.
■ getMinutesToDeparture() returning an int.
■ setMinutesToDeparture(int minutesToDeparture) returning void.

2. Class: DomesticFlight (Child Class)
This class extends the Flight base class.
● Attributes:
○ An integer representing the terminal zone.
● Constructor:
○ Takes three parameters: int flightCode, int minutesToDeparture, and int terminalZone. You must use the super() keyword to initialize the inherited attributes.
● Methods:
○ getTerminalZone() returning an int.

3. Class: InternationalFlight (Child Class)
This class also extends the Flight base class.
● Attributes:
○ An integer representing the time required for customs clearance (customsTime).
(Note: This value can be exactly 0).
● Constructor:
○ Takes three parameters: int flightCode, int minutesToDeparture, and int customsTime. Again, utilize the super() keyword for the parent attributes.
● Methods:
○ getTotalMinutesToDeparture() returning an int. This method calculates and returns the total time before the plane actually leaves.
○ Crucial Rule: Do not calculate and store this total time in the constructor! You must dynamically calculate it (by adding the inherited minutesToDeparture to the customsTime) every time this method is called. This ensures the total time remains accurate even if the inherited minutes are changed later via a setter method.

The Manager Class: Airport
This class is the core engine of your simulation. It manages two separate fixed-size arrays representing the physical gates at the airport: one for domestic flights and one for international flights.
● Constructor:

○ Takes two parameters: int domesticCapacity and int intlCapacity. It initializes the internal arrays using these capacities. (Note: The system should not crash if an airport is built with a capacity of 0!).

● Methods:

○ assignGate(DomesticFlight df) returning void: This is an overloaded method. It iterates through the domestic gates array from the beginning (index 0). It finds the first empty spot (a null index), assigns the provided flight object to it. If the array is completely full, it throws it throws the provided custom DomesticFullException with the index of the gate holding the flight scheduled to leave the soonest (the minimum minutes to departure).

○ assignGate(InternationalFlight itf) returning void: This is the second version of the overloaded method. It performs the exact same logic as above, but operates exclusively on the international gates array and uses the InternationalFullException.

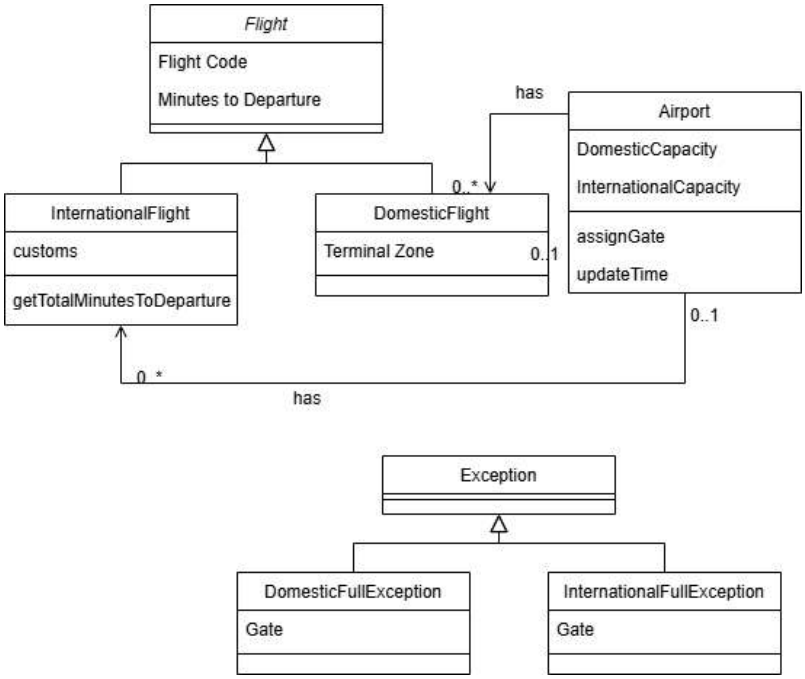
○ updateTime(int minutesPassed) returning void:

This simulates the passage of time or delays. It iterates through both arrays. For every active flight, it subtracts the minutesPassed from the flight's current minutes to departure. Note: if minutesPassed is negative, that is how we will simulate a delay to all planes.

■ Departure Condition: If a flight's remaining time drops to exactly 0 or below, it has departed! You must free up the gate by setting that array index back to null.

■ Safety Check: Make sure your loops safely skip over already empty (null) gates without throwing a NullPointerException.

NOTE: the provided Exception classes should be children of the java Exception class as shown in the example UML diagram below describing the system.




# Requested files

## AirportManager.java

```java
public class AirportManager {
    public static void main(String[] args) {

        Airport airport = new Airport(2, 2);

        DomesticFlight d1 = new DomesticFlight(101, 50, 1);
        DomesticFlight d2 = new DomesticFlight(102, 20, 2);
        DomesticFlight d3 = new DomesticFlight(103, 10, 3);
        DomesticFlight d4 = new DomesticFlight(104, 15, 4);

        InternationalFlight i1 = new InternationalFlight(201, 40, 15);
        InternationalFlight i2 = new InternationalFlight(202, 45, 10);
        InternationalFlight i3 = new InternationalFlight(203, 20, 10);

        try {
            airport.assignGate(d1);
            airport.assignGate(d2);
            airport.assignGate(d3); // Exception
            airport.assignGate(d4); // Should not even attempt this
        } catch (Exception ex) {
            System.out.println(ex); // DomesticFullException: Next available gate - 1
        }

        try {
            airport.assignGate(i1);
            airport.assignGate(i2);
            airport.assignGate(i3); // Exception
        } catch (Exception ex) {
            System.out.println(ex); // InternationalFullException: Next available gate - 0
        }

        System.out.println("International Flight Total Time (should be 55): " +
                i1.getTotalMinutesToDeparture());

        airport.updateTime(25);
        System.out.println("International Flight Total Time (should be 30): " +
                i1.getTotalMinutesToDeparture());

        airport.updateTime(-10);
        System.out.println("International Flight Total Time (should be 40): " +
                i1.getTotalMinutesToDeparture());

        System.out.println("Domestic Flight Time (should be 35): " +
                d1.getMinutesToDeparture());
    }
}
// You should not modify anything above this line

class DomesticFullException {
   private int gate;

   public DomesticFullException(int gate) {
      this.gate = gate;
   }

   public String toString() {
      return "DomesticFullException: Next available gate - " + gate;
   }
}

class InternationalFullException {
   private int gate;

   public InternationalFullException(int gate) {
      this.gate = gate;
   }

   public String toString() {
      return "InternationalFullException: Next available gate - " + gate;
   }
}
```

VPL